High Performance Computing for Artificial Intelligence

applications in physics

performance and benchmarking

purpose of benchmarking

- evaluate hardware and software under specified conditions
- compare systems using controlled, reproducible measurements
- identify bottlenecks and optimize configurations
- guide procurement decisions for new HPC systems

performance metrics

- **time-to-solution** (tts): execution time of a computation
- floating-point operations per second (**flop/s**)
- **memory bandwidth**: data transfer rate between CPU and memory
- **latency**: delay in communication or data access

peak vs. sustained performance:

- **peak** = theoretical maximum (e.g., flop/s from hardware specs)
- **sustained** = what applications actually achieve

reliable measurements

- validate the result before comparing performance
- measure **elapsed time**; summed CPU time can exceed it when several threads run concurrently
- define the interval: whole program or selected phase; state whether setup and input/output are included
- repeat runs; report a typical value and its variability
- record input, compiler/modules, hardware, process/thread layout

profiling

timing measures **how long**; profiling identifies **where** time is spent

```
perf record -e cpu-clock:u -F 99 -g ./program
perf report --stdio
```

- run on a compute node; compile with optimization and debugging symbols
- **Self**: samples in the function; **Children**: includes called functions
- percentages are relative to each run: compare elapsed times after a change

theoretical peak performance

of a compute node, in double precision

$$R_{\text{peak}} = N_{\text{sockets}} \times N_{\text{cores per socket}} \times N_{\text{operations per cycle}} \times R_{clock}$$

example: server with $N_{\text{sockets}} = 2$ AMD EPYC 7313 processors

- $N_{\text{cores per socket}} = 16$
- $N_{\text{operations per cycle}} = 16$ in double precision
  two AVX2 (256-bit) fused multiply-add (FMA) units
- $R_{clock} = 3\,\text{GHz}$

theoretical peak performance

exercise: what is the R_{peak} of a Raspberry Pi 5?

- the command `lscpu` reports details about the processor model
- information on $N_{\text{operations per cycle}}$ for various processors can be found here

HPC benchmark examples

- **High Performance Linpack (HPL)**:
  dense linear algebra, solves the matrix equation $Ax = b$
- **STREAM**:
  synthetic benchmark, estimates sustained memory bandwidth
- **High Performance Conjugate Gradient (HPCG)**:
  sparse computation, memory access and communication
- **Graph500**: graph traversal, data-intensive load
- **MLPerf HPC**: "A Holistic Benchmark Suite for Scientific Machine Learning on HPC Systems"

High Performance Linpack (HPL)

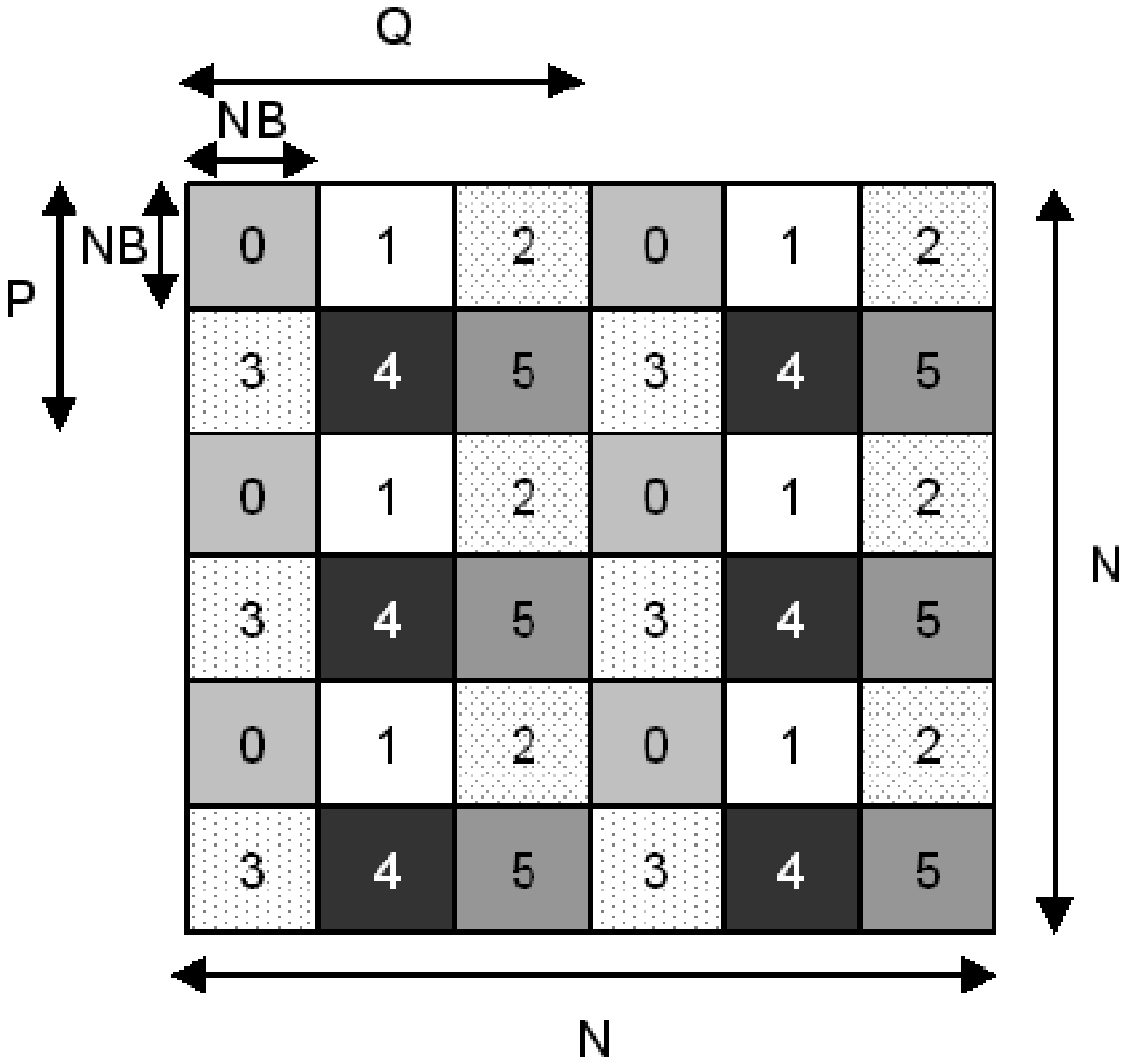
- LINPACK: a Fortran library to analyze and solve linear equations, long superseded by LAPACK
- the original LINPACK benchmark measures the performance of a system by solving a dense $n \times n$ linear system $Ax = b$
- HPL is the portable, distributed-memory implementation of the benchmark
- the LINPACK benchmark family underlies the TOP500 ranking
- HPL: portable implementation written in C

HPL

the latest sources can be downloaded from the HPL website and compiled against implementations of MPI and the Basic Linear Algebra Subprograms (BLAS)

- on WILSON, use `module spider HPL` to find the required compiler and MPI modules before loading HPL
- a BLAS implementation tuned for the hardware is usually used

the `xhpl` executable reads the 31-line input file `HPL.dat` from the current directory;
the main parameters to check are **N**, **NB**, **P**, and **Q**



params in the `HPL.dat`

- NB: the block size e.g. 192
- P×Q is the number of MPI processes
- N: problem size prefer NB multiples

$$N \approx \sqrt{\frac{\beta \cdot \text{total memory}}{\texttt{sizeof(double)}}}$$

with $\beta \approx 0.8$

[img source]

tune HPL

- HPL.dat tuning tool, with example starting configurations:
  - first check on WILSON: P=Q=2, `OMP_NUM_THREADS=4`, NB=192, N=12288 (matrix: 1.125 GiB)
  - larger case on a 2×8-core node with 64 GiB RAM: same layout and NB, N=82560
  - Raspberry Pi 5 with 4 GiB RAM: P=Q=1, `OMP_NUM_THREADS=4`, NB=192, N=20352
- validate the small case before increasing N
- P×Q×`OMP_NUM_THREADS` = the number of requested cores

HPL scaling

let p be the number of MPI processes; keep threads per process fixed

- **strong scaling:** keep N fixed, increase p
- **memory scaling:** increase $N_p \propto \sqrt{p}$
  - matrix storage per process: $N_p^2/p \approx \text{constant}$
  - arithmetic work per process: $N_p^3/p \propto \sqrt{p}$
- constant memory per process is **not ordinary weak scaling**
- larger memory-scaled cases can take much longer;
  estimate the cost from smaller runs

HPL limitations

- it is one number to characterize a complete system
- high operational intensity,
  not representative of many scientific workloads
- $O(\mathrm{N}^3)$ computation and $O(\mathrm{N}^2)$ storage
  - peak performance requires large fraction of system memory
  - runs on the largest distributed-memory systems can take days

High Performance Conjugate Gradient

or the HPCG benchmark

- designed to **complement HPL**
- solves the **sparse** linear system of a 3D Laplace equation, 27-point stencil, using the **conjugate gradient** method
  - a Krylov subspace solver algorithm
- memory-access pattern vastly different from HPL, more realistic
- top500.org publishes a list also for HPCG results

HPCG parameters

`xhpcg` looks for `hpcg.dat` in the current folder

```
HPCG benchmark input file
Sandia National Laboratories; University of Tennessee, Knoxville
104 104 104
60
```

- first two lines are unused, can be used for user comments
- third line specifies the 3D problem size local to the MPI process
- fourth line specifies how long the benchmark should run in seconds
  1800 s is required for official results; here 60 s is enough

HPCG problem size

run time is configured: compare validated flop/s, not time to solution

- `hpcg.dat` sets the **local problem size** per MPI process
  $\Rightarrow$ fixed local dimensions give a weak-scaling scenario
- N_x, N_y and N_z: at least 16; use multiples of 8
- memory used is about $896\ \mathrm{B} \times N_x N_y N_z$;
  target at least 25% of system memory, for example:
  - $N_x = N_y = N_z = 104$: about 0.94 GiB per MPI process
  - $N_x = N_y = N_z = 160$: about 3.42 GiB per MPI process

microbenchmarks

capture a single performance characteristic,
e.g. the Ohio State University (OSU) Micro-Benchmarks for communication, including MPI benchmarks

miniapplications

are smaller versions of real-world scientific applications
and generally report time to solution for a standard problem size,
using proven code, e.g. miniFE (finite elements), Grid (lattice QCD)